

ML chip hw2 report

In HW2, we are asked to implement Alexnet. The communication between each module should be realized by `sc_fifo`, `sc_signal`, `sc_buffer`. In my implementation, each layer in the Alexnet is a pipeline stage, which means there are 20 stages in the Alexnet. The final output will be output at the 20th cycle. The simulation is terminated by `sc_stop` at time = 80ns.

Simulation result

signal

```

14:34 mlchip115@ee21[~/hw2/sc_signal]$ make exe
g++ -g -I . -I /RAID2/COURSE/mlchip/mlchipTA01/systemc-
-lsystemc-2.3.3 -Wl,-rpath,/RAID2/COURSE/mlchip/mlchipTA
./run ../../data/cat.txt

SystemC 2.3.3-Accellera --- Mar  2 2024 23:27:20
Copyright (c) 1996-2018 by all Contributors,
ALL RIGHTS RESERVED
Class 1: Egyptian cat          val = 20.2067
Class 2: tabby                 val = 16.1368
Class 3: tiger cat            val = 15.7338
Class 4: lynx                  val = 14.7909
Class 5: plastic bag          val = 14.4119

Info: /OSCI/SystemC: Simulation stopped by user.
./run ../../data/dog.txt

SystemC 2.3.3-Accellera --- Mar  2 2024 23:27:20
Copyright (c) 1996-2018 by all Contributors,
ALL RIGHTS RESERVED
Class 1: golden retriever      val = 16.5945
Class 2: otterhound            val = 15.5697
Class 3: Sussex spaniel       val = 15.3619
Class 4: bloodhound           val = 15.0027
Class 5: cocker spaniel       val = 14.5932

Info: /OSCI/SystemC: Simulation stopped by user.

```

Buffer

```

14:46 mlchip115@ee21[~/hw2/sc_buffer]$ make exe
g++ -g -I . -I /RAID2/COURSE/mlchip/mlchipTA01/systemc-2.3.3/
-lsystemc-2.3.3 -Wl,-rpath,/RAID2/COURSE/mlchip/mlchipTA01/sys
./run ../../data/cat.txt

SystemC 2.3.3-Accellera --- Mar  2 2024 23:27:20
Copyright (c) 1996-2018 by all Contributors,
ALL RIGHTS RESERVED

Cycle 20
Class 1: Egyptian cat          val = 20.2067
Class 2: tabby                 val = 16.1368
Class 3: tiger cat            val = 15.7338
Class 4: lynx                 val = 14.7909
Class 5: plastic bag          val = 14.4119

Info: /OSCI/SystemC: Simulation stopped by user.
./run ../../data/dog.txt

SystemC 2.3.3-Accellera --- Mar  2 2024 23:27:20
Copyright (c) 1996-2018 by all Contributors,
ALL RIGHTS RESERVED

Cycle 20
Class 1: golden retriever      val = 16.5945
Class 2: otterhound            val = 15.5697
Class 3: Sussex spaniel       val = 15.3619
Class 4: bloodhound           val = 15.0027
Class 5: cocker spaniel       val = 14.5932

Info: /OSCI/SystemC: Simulation stopped by user.

```

FIFO

```

Conv2d.h DEBUG.h Linear.h Makefile Monitor.h Pattern.h Pool.h ReLU.h main.cpp run
15:30 mlchip115@ee21[~/hw2/sc_fifo]$ make exe
g++ -g -I . -I /RAID2/COURSE/mlchip/mlchipTA01/systemc-2.3.3/include -L . -L /RAID2/COURSE/mlc
hip/mlchipTA01/systemc-2.3.3/lib-linux64 -o run main.cpp -lsystemc-2.3.3 -Wl,-rpath,/RAID2/COUR
SE/mlchip/mlchipTA01/systemc-2.3.3/lib-linux64
./run ../../data/cat.txt

SystemC 2.3.3-Accellera --- Mar  2 2024 23:27:20
Copyright (c) 1996-2018 by all Contributors,
ALL RIGHTS RESERVED
Class 1: Egyptian cat          val = 20.2067
Class 2: tabby                 val = 16.1368
Class 3: tiger cat            val = 15.7338
Class 4: lynx                  val = 14.7909
Class 5: plastic bag          val = 14.4119
./run ../../data/dog.txt

SystemC 2.3.3-Accellera --- Mar  2 2024 23:27:20
Copyright (c) 1996-2018 by all Contributors,
ALL RIGHTS RESERVED
Class 1: golden retriever      val = 16.5945
Class 2: otterhound            val = 15.5697
Class 3: Sussex spaniel       val = 15.3619
Class 4: bloodhound           val = 15.0027
Class 5: cocker spaniel       val = 14.5932
15:54 mlchip115@ee21[~/hw2/sc_fifo]$

```

Challenge encountered

When implementing `sc_signal` and `sc_buffer`, there was not much problem since I simply replaced the original interface from HW1 for `sc_signal_in_if/sc_buffer_in_if`. But when implementing `sc_fifo`, I forgot the fact that one fifo cannot be read multiple times within one cycle. As a result, the output stuck at a incorrect value but the simulation went on and on.

The problem here is that in the module `MaxPool2d`, some of the data is access multiple times, and if we directly read out data from the fifos, the run function cannot complete its execution at the expected stage, and the execution goes on and on and on since everytime a fifo is read twice, the execution of run function pauses, and continues when the fifo data is once again available.

To tackle this, I create an array to store the data from `in_data` so that every fifo is accessed only once.

```

void MaxPool2d::run()
{
    while (true)
    {
        if (DEBUG)

```

```

        cout << "run " << name << endl;

int out_wid = in_wid / stride;

// a 3D vector to store the input data
// Every fifo is read only once
vector<vector<vector<double>>> in_array(in_ch, vecto
for (int i = 0; i < in_ch; i++)
{
    for (int j = 0; j < in_wid; j++)
    {
        for (int k = 0; k < in_wid; k++)
        {
            double temp;
            in_data[i][j][k]->read(temp);
            in_array[i][j][k] = temp;
        }
    }
}

for (int i = 0; i < out_wid; i++)
{
    for (int j = 0; j < out_wid; j++)
    {
        for (int k = 0; k < in_ch; k++)
        {
            double max_val = 0;
            for (int l = 0; l < kernel_size; l++)
            {
                for (int m = 0; m < kernel_size; m++)
                {
                    double temp;
                    temp = in_array[k][i * stride +
                    // if in_data[k][i * stride + l
                    // the program will stop once o
                    max_val = max(max_val, temp);
                }
            }
        }
    }
}

```

```

        }
    }
    out_data[k][i][j]->write(max_val);
}
}
}

if (DEBUG)
    cout << "write " << name << endl;
wait();
}
}

```

Observation: fifo version only takes 1 cycle

In the fifo implementation, read() function is used to read out the fifo data. This will result in the fact that only 1 clk period is required for the program to produce the final result. The screenshot of output message is provided to help explain the whole process.

At the beginning of the simulation, the thread function of all the modules were called, which in the below picture indicated by "run ...(module name)". But since all fifos were empty, all those functions except Pattern::run() paused at sc_fifo::read() until the sc_fifo::write() in Pattern::run() was done. The execution of run() for conv1 was later resumed since its input fifo is filled by Pattern::run(), and thus successfully write data into the output of conv1(indicated by the message "write ... (module name)" , causing the run() in the next stage to continue its execution, so on and so forth. Within the end of the clk cycle, all modules had completed the execution of its run(), and thus gave us the final answer. This is why the computation only takes 1 cycle.

```

start simulating
run Pattern
write Pattern
run conv1
run ReLU_3D
run pool1
run conv2
run ReLU_3D
run pool2
run conv3
run ReLU_3D
run conv4
run ReLU_3D
run conv5
run ReLU_3D
run pool5
run flatten
run fc6
run ReLU_1D
run fc7
run ReLU_1D
run fc8
write conv1
write relu1
write pool1
write conv2
write relu2
write pool2
write conv3
write relu3
write conv4
write relu4
write conv5
write relu5
write pool5
write flatten
write fc6
write fc7
write fc8
Class 1: Egyptian cat          val = 20.2067
Class 2: tabby                 val = 16.1368
Class 3: tiger cat            val = 15.7338
Class 4: lynx                 val = 14.7909
Class 5: plastic bag          val = 14.4119
end simulating

```

If we wish to simulate the exact same behavior in `sc_signal` version, we will have to use `sc_fifo::nb_read` and pause the `run()` function whenever the input is unavailable.